

NOTE TO READER. This document is a **presentation draft prepared for scholarly discussion purposes only**. It is not a submitted or peer-reviewed manuscript. It describes, in academic paper format, the architecture and analytical capabilities of the Synapse IDE Urban Analytics Workbench as currently implemented. All technical details are derived directly from the source code of the software system.

Synapse IDE: Urban Analytics Workbench

**A Browser-Native, Three-Panel Computational Environment
for Reproducible Spatial Analysis, AI-Assisted Planning,
and Interactive Urban Simulation**

Mustafa Raşit Şahin

Instructor, Department of City and Regional Planning
Middle East Technical University (METU), Ankara, Türkiye
Financial Affairs Specialist, Altındağ Belediyesi, Ankara
rsahin@metu.edu.tr | github.com/mustafaras
ORCID: 0009-0001-4809-3950

Presentation draft — May 8, 2026

Abstract

The integration of spatial statistics, urban simulation, geospatial data management, three-dimensional visualisation, and large-language-model (LLM) assistance into a single reproducible workflow represents an unmet need in computational planning practice. Existing toolchains assemble these capabilities across heterogeneous environments—GIS platforms, Python notebooks, simulation software, and separate AI services—imposing configuration overhead, breaking the analytical chain from data to publication, and limiting accessibility for researchers and practitioners in resource-constrained contexts.

This paper describes **Synapse IDE: Urban Analytics Workbench**, an open-source, browser-native research environment implemented in TypeScript and React. The system is organised into three tightly integrated panels: (i) a **Coder** panel providing a Monaco-based code editor with multi-provider AI streaming and an apply-plan pipeline; (ii) an **Urban**

Analytics panel providing structured eight-section research workflows, eight analytical flow templates, an indicator catalogue spanning eight thematic groups, dashboard composition, and AI-assisted reporting; and (iii) a **Map Explorer** panel providing a reactive layer stack, GeoJSON loading, multi-format export, and a bridge interface between the code and analytics panels. The underlying engine layer implements geographically weighted regression (GWR) with AICc-optimal bandwidth selection, Local Indicators of Spatial Association (LISA) with Benjamini–Hochberg correction, geostatistics, cellular automata, agent-based modelling, facility siting optimisation, VoxCity three-dimensional building extrusion with solar simulation, and a domain-aware retrieval-augmented generation (RAG) corpus. A domain-specific TypeScript type system spanning eight urban scales, twenty data-source connectors, and approximately sixty thematic tags provides controlled vocabulary across all three panels. Reproducibility is enforced through deterministic seeding, manifest-backed artefact traceability, and a background-worker pool with cancellation and progress management. The workbench is designed to lower the computational planning barrier for research and practice in data-sparse urban contexts.

Keywords: computational urban planning; spatial statistics; GWR; LISA; agent-based modelling; LLM orchestration; browser-native GIS; reproducible science; urban analytics; three-dimensional visualisation

0 Contents

1	Introduction	2
2	Background and Related Work	2
2.1	Computational planning environments	2
2.2	AI assistance in spatial analysis	3
2.3	Reproducibility in spatial research	3
3	Architecture and Domain Type System	3
3.1	Overall architecture	3
3.2	Domain type system	3
3.3	Background Worker Pool	4
4	Three-Panel User Interface	4
4.1	Coder Panel	5
4.2	Urban Analytics Panel	5
4.2.1	Analytical Flow Templates	5
4.2.2	Indicator Catalogue	6
4.3	Map Explorer Panel	7
5	Spatial Statistics Engine	7
5.1	Geographically Weighted Regression	7
5.2	Local Indicators of Spatial Association	8
5.3	Geostatistics and Regression	8
6	Simulation Engine	8
6.1	Cellular Automata	8
6.2	System Dynamics	9
6.3	Agent-Based Modelling	9
6.4	Facility Siting Optimisation	9
7	Three-Dimensional Visualisation: VoxCity	9
8	AI Orchestration and Retrieval-Augmented Generation	10
8.1	Multi-Provider LLM Orchestration	10
8.2	Retrieval-Augmented Generation	10
9	Reproducibility Design	10
10	Discussion and Limitations	11

10.1 Contributions	11
10.2 Current limitations	11
10.3 Future directions	11
11 Conclusion	11

1 Introduction

Urban planning and spatial analysis are methodologically rich but operationally fragmented. Practitioners and researchers routinely combine OpenStreetMap geodata, satellite imagery, spatial econometrics, simulation, and increasingly LLM assistance—but almost invariably across separate, poorly integrated tools. QGIS and ArcGIS handle cartography but not AI assistance. Python notebooks handle statistics but impose configuration overhead and provide no domain-structured analytical templates. Simulation platforms such as UrbanSim [Waddell, 2002] are model-specific. The result is that the analytical chain from raw data to published output is opaque, difficult to audit, and hard to replicate.

This fragmentation is particularly costly in resource-constrained institutional contexts, where practitioners cannot maintain complex toolchain stacks. A browser-native environment requiring no local installation and providing integrated spatial computation, simulation, AI assistance, and publication-grade export could substantially lower this barrier.

Synapse IDE: Urban Analytics Workbench addresses this problem through a three-panel architecture that keeps the computational, analytical, and cartographic environments simultaneously visible and interoperable. Its central design commitments are: (1) domain specificity—a structured type system rather than an unstructured notebook; (2) browser-nativity—no server dependency for core analytical functions; (3) AI integration—multi-provider LLM streaming embedded in the code editor and analytical flows; (4) reproducibility—deterministic seeding, manifest artefact tracing, and apply-plan history.

The remainder is organised as follows. Section 2 situates the workbench relative to existing tools. Section 3 describes the overall architecture and domain type system. Section 4 describes the three-panel user interface in detail. Sections 5–8 describe the engine subsystems. Section 9 covers reproducibility design. Section 10 discusses limitations and future directions.

2 Background and Related Work

2.1 Computational planning environments

The GIS tradition—represented by ArcGIS and QGIS—provides robust spatial operations and cartography, but treats statistical modelling and simulation as secondary concerns handled through plug-ins or external scripts. The Python geospatial ecosystem (GeoPandas, PySAL, OSMnx [Boeing, 2017], Rasterio, NetworkX) offers methodological richness but no domain structure: analytical choices are implicit in code rather than explicit in workflow templates.

Jupyter and Observable notebooks improve reproducibility relative to ad hoc scripting but provide no controlled domain vocabulary, no integrated AI assistance, and no structured document or reporting export. Dedicated simulation platforms (UrbanSim, SLEUTH, NetLogo) are model-specific and not general-purpose analytical environments. Commercial integrated platforms such as Esri Urban and Cityzenith offer 3D urban analysis but impose licensing constraints and sacrifice source transparency, making them unsuitable for open research contexts.

2.2 AI assistance in spatial analysis

The integration of large language models into spatial analysis workflows is nascent. CityGPT [Feng et al., 2025] and UrbanGPT [Li et al., 2023] explore language grounding for urban tasks but are inference systems rather than analyst workbenches. GeoAI approaches embed spatial reasoning in model architectures [Mai et al., 2023] but do not provide the statistical computation, simulation, and artefact management that an end-to-end workflow requires. Retrieval-augmented generation [Lewis et al., 2020] has been applied to scientific literature and domain-specific question answering, but its integration with spatial computation remains underdeveloped.

2.3 Reproducibility in spatial research

Reproducibility in geographic and urban research has been identified as a structural challenge [Singleton et al., 2016]. The analytical chain from raw data to published results is typically poorly documented, and the computational environment is rarely pinned. Manifest-based artefact management, deterministic seeding, and environment pinning—standard in software engineering—have been adopted unevenly in spatial research. Synapse makes artefact traceability a first-class design constraint at the architectural level.

3 Architecture and Domain Type System

3.1 Overall architecture

Synapse IDE is implemented as a browser-native single-page application using TypeScript, React, and Vite. Four architectural layers structure the system:

1. **Engine layer:** spatial statistics, simulation, GPU computation, and streaming connectors, executing in Web Workers for UI non-blocking.
2. **Service layer:** AI orchestration, RAG corpus, data pipeline, reporting, and reinforcement-learning routing.
3. **State layer:** Zustand-based reactive stores for workspace, AI configuration, background tasks, spatial index, and streaming runtime.
4. **Interface layer:** three-panel UI (Coder, Urban Analytics, Map Explorer) with shared-state bridges.

3.2 Domain type system

The `urbanAnalytics/lib/types.ts` module defines a domain-specific type system serving as the single source of truth across all three panels. Table 1 summarises the principal enumerated types.

Type	Cardinality	Members (selected)
UrbanScale	8	parcel, block, neighborhood, district, city, metropolitan, regional, national
DataSource	20	osm, census, sentinel, landsat, gtfs, iot_sensor, lidar, cadastral, cityjson, google_maps, custom
GeoDataFormat	20	geojson, arrow, geoparquet, shapefile, cog, gtfs, osm_pbf, cityjson, ifc, 3dtiles
UrbanTag	≈60	mobility, transit, land_use, zoning, environment, demography, economics, governance, ...
SessionType	6	baseline, field_survey, desk_study, stakeholder, scenario_modeling, monitoring

Table 1: Principal enumerated types in the domain type system (`urbanAnalytics/lib/types.ts`). These types drive UI components, constrain query routing, and provide structured metadata for RAG expansion and evidence export.

3.3 Background Worker Pool

Computationally intensive tasks are offloaded via a typed `BackgroundWorkerPool`. Each submitted job carries typed input/output contracts, a cancellation token, and a progress callback. The pool manages slot recycling, timeout enforcement, and a subscribe-pattern snapshot distribution that allows the UI to display per-job status without polling.

4 Three-Panel User Interface

The interface is organised into three tightly integrated panels sharing reactive state through Zustand stores and two bridge connections: *Map Bridge* (Coder ↔ Map Explorer) and *Urban Bridge* (Coder ↔ Urban Analytics). Table 2 provides a comparative overview.

Coder Panel	Urban Analytics Panel	Map Explorer Panel
Activity Rail (7 items)	Tabs: Projects / New Project / Methods / Education / Report / Workflows / Dashboard / Toolbox	Reactive layer stack
Monaco code editor	Structured sections (7 + sub-sections)	GeoJSON / CSV loader
Multi-provider AI stream	8 analytical flow templates	Multi-format export
Apply-plan pipeline	Indicator catalogue (8 groups)	View state & bounds
Plan history	Dashboard builder	Map Bridge UI
Urban Bridge panel	AI-assisted reporting	Google Maps layers
<i>Shared state:</i> workspace store, spatial index, streaming runtime, AI configuration		

Table 2: Three-panel layout of Synapse IDE. Panels communicate via two bridge channels (Map Bridge and Urban Bridge) and shared Zustand stores.

4.1 Coder Panel

The Coder panel provides a Monaco-based code editor with urban-domain extensions. Seven activity-rail items provide persistent navigation: Settings, Explorer, Outline, Search, Plan History, Map Bridge, and Urban Bridge. The AI panel (`SynapseCoreAIPanel`) operates a finite-state machine governing transitions between idle, streaming, and error states. At each conversational turn a context bundle is assembled from the active workspace:

$$\mathcal{C}_t = \{ \text{file path, file content, cursor selection, diagnostics summary, workspace metadata} \} \quad (1)$$

This bundle is injected into the system prompt, grounding model responses in the specific code and analytical context without requiring the user to restate it.

The *apply-plan pipeline* implements a safe-edit workflow: the AI proposes a structured changeset (insertions, deletions, replacements) which is previewed and then accepted or rejected. All accepted changesets are recorded in `useApplyHistoryStore` with before/after state snapshots, enabling selective rollback and cross-revision comparison.

4.2 Urban Analytics Panel

The Urban Analytics panel provides a seven-section structured research document with subsections covering Demographics, Land Use, Built Form, Transport, Environment, Spatial Statistics, Multivariate Analysis, Network Analysis, Remote Sensing, GeoAI & ML, Simulations, Morphology Metrics, Accessibility Metrics, Environmental Metrics, Equity Metrics, Design Proposals, Policy Recommendations, and Scenario Comparison.

4.2.1 Analytical Flow Templates

Eight pre-structured analytical flows decompose each workflow into ordered step configurations (Table 3).

ID	Template	Key steps
1	Site Suitability Analysis	Criteria, data layers, weighting (equal / AHP / manual), constraint masking, Monte Carlo sensitivity
2	Accessibility Analysis	Mode (walk/cycle/transit/drive), threshold, POI categories, equity disaggregation, isochrones
3	Vulnerability Assessment	Hazard type, exposure, sensitivity, adaptive capacity; Risk = $H \times E \times S - C$
4	Composite Indicator Builder	OECD/JRC style: normalisation, weighting, aggregation (additive/geometric), Sobol diagnostics
5	Scenario Comparison	Baseline definition, 2–4 alternatives, metric selection, side-by-side output
6	Urban Morphology Clustering	Multivariate k -means, morphotype assignment, layer publication to Map Explorer
7	YOLO-Nano Object Detection	Tiled inference, class-filtered GeoJSON, review-ready run archival
8	VoxCity / CityJSON / Sunlight	Building extrusion (LOD), CityJSON v2.0 import, solar irradiance computation

Table 3: Analytical flow templates in the Workflows tab. Each flow is decomposed into sequential step configurations stored in `FLOW_DEFINITIONS` (`centerpanel/Flows/flowTypes.ts`).

4.2.2 Indicator Catalogue

The Toolbox tab provides a structured catalogue of urban performance indicators organised into eight thematic groups (Table 4).

Group	Scope
Transport & Mobility	Network throughput, access, safety, modal balance
Energy & Climate	Operational energy, carbon, climate-response metrics
Urban Form & Landscape Ecology	Density, fabric, landscape-structure, morphological form
Social Infrastructure & Liveability	Equity, amenity, public-space quality
Water & Infrastructure	Consumption, runoff, utility reliability, grey-green capacity
Governance & Innovation	Permitting, participation, data maturity, institutional readiness
Heritage & Culture	Built and intangible heritage continuity
Pandemic Resilience	Essential services, density risk, digital equity

Table 4: Indicator catalogue groups in the Toolbox tab (`features/urbanAnalytics/indicators`). Each indicator carries a traceability record linking it to its source dataset and computation method.

4.3 Map Explorer Panel

The Map Explorer panel provides a reactive, layer-stack-based cartographic environment through four core hooks:

- **useMapState**: view-state management (centre, zoom, bearing, pitch) with bounds tracking
- **useLayerStack**: typed layer configuration (point, line, polygon, raster, heatmap, cluster, 3D extrusion) with per-layer visibility, opacity, and style control
- **useGeoJSONLoader**: in-memory-cached loader supporting GeoJSON and CSV (with automatic lat/lon column detection and feature conversion)
- **useMapExport**: export pipeline to GeoJSON, GeoTIFF, PDF (with metadata injection), and PNG

The Map Bridge (**usePanelBridgeStore**) enables bidirectional feature passing: a clustering result computed in the Urban Analytics panel publishes directly as a new Map Explorer layer; a feature selected on the map injects context into the AI panel’s next prompt. The **MapExplorerModal** provides a full-screen cartographic view accessible from any panel.

5 Spatial Statistics Engine

5.1 Geographically Weighted Regression

The GWR implementation follows Fotheringham et al. [2002]. The local estimator at location v_i is:

$$\hat{\beta}(v_i) = \left(\mathbf{X}^\top \mathbf{W}(v_i) \mathbf{X} \right)^{-1} \mathbf{X}^\top \mathbf{W}(v_i) \mathbf{y} \quad (2)$$

where $\mathbf{W}(v_i) = \text{diag}(w_{i1}, \dots, w_{in})$ is the kernel-weight matrix. Two kernels are implemented:

$$w_{ij}^{\text{bisquare}} = \left[1 - \left(\frac{d_{ij}}{h} \right)^2 \right]^2 \cdot \mathbf{1}(d_{ij} < h) \quad (3)$$

$$w_{ij}^{\text{Gaussian}} = \exp \left(-\frac{d_{ij}^2}{2h^2} \right) \quad (4)$$

where d_{ij} is the Euclidean distance between observations on projected coordinates and h is the bandwidth. When bandwidth is unspecified, it is selected by minimising the corrected Akaike information criterion [Hurvich et al., 1998]:

$$\text{AICc}(h) = 2n \ln \hat{\sigma}(h) + n \ln(2\pi) + n \cdot \frac{n + \text{tr}(\mathbf{S})}{n - 2 - \text{tr}(\mathbf{S})} \quad (5)$$

where $\mathbf{S}$ is the hat matrix and $\hat{\sigma}^2 = \mathbf{y}^\top (\mathbf{I} - \mathbf{S})^\top (\mathbf{I} - \mathbf{S}) \mathbf{y} / (n - \text{tr}(2\mathbf{S} - \mathbf{S}^\top \mathbf{S}))$. Minimisation uses golden-section search with configurable tolerance (default: 0.1% of range). Total computational complexity is $O(n^2 p^2)$ per bandwidth candidate.

Per-location outputs—organised as `GWRParameterSurface` objects—include coefficient estimates, standard errors, t -statistics, local R^2 , and fitted values.

5.2 Local Indicators of Spatial Association

Local Moran’s I [Anselin, 1995] for observation i is:

$$I_i = \frac{(x_i - \bar{x})}{m_2} \sum_j w_{ij} (x_j - \bar{x}), \quad m_2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2 \quad (6)$$

Statistical significance is assessed via conditional permutation (B realisations) with the Mulberry32 PRNG at stored seed s :

$$p_i^{\text{perm}} = \frac{|\{b \in [1, B] : I_i^{(b)} \geq I_i\}|}{B} \quad (7)$$

Multiple testing correction applies the Benjamini–Hochberg procedure [Benjamini & Hochberg, 1995] at false discovery rate q :

$$k^* = \max \left\{ k : p_{(k)} \leq \frac{k}{m} q \right\} \quad (8)$$

Observations are classified into HH, LL, HL, LH, and non-significant clusters. Getis-Ord G_i^* [Getis & Ord, 1992] hotspot analysis and bivariate LISA are also implemented.

5.3 Geostatistics and Regression

The geostatistics module provides variogram estimation, ordinary kriging, simple kriging, and inverse distance weighting, with leave-one-out cross-validation. The regression engine supports OLS with spatial diagnostics (Moran’s I on residuals, Breusch-Pagan test, spatial lag and spatial error models), panel data with Newey-West standard errors, quantile regression, and multiscale GWR (MGWR).

6 Simulation Engine

6.1 Cellular Automata

The CA module implements mask-aware, regime-conditioned urban growth simulation. Let $\mathbf{L}_t$ denote the gridded land-state at time t . Transition probability for cell c is:

$$P(\ell_c^{(t+1)} = 1 \mid \mathbf{L}_t, R_t, M_c) = M_c \cdot \sigma(\boldsymbol{\theta}_{R_t}^\top \phi(\mathcal{N}(c))) \quad (9)$$

where $\sigma(\cdot)$ is the logistic function, ϕ is a neighbourhood feature vector, R_t is the planning-regime indicator, and $M_c \in \{0, 1\}$ is the spatial feasibility mask. Three scenario families are generated by perturbation of θ_{R_t} . Validation follows LOTO and BHO protocols, reporting SSIM, MAE, and residual Moran's I .

6.2 System Dynamics

Stock-and-flow models are integrated via Euler integration with step size Δt and adaptive error control. A model $\mathcal{M} = (\mathbf{S}, \mathbf{F}, \mathbf{A}, \theta)$ consists of named stocks, flows, auxiliaries, and parameter tables. Uncertainty intervals are produced via Monte Carlo parameter sweeps.

6.3 Agent-Based Modelling

The ABM framework supports typed agent initialisation from geospatial datasets and typed update functions. Replicator dynamics are implemented for population-level diffusion:

$$\dot{x}_k = x_k(f_k(\mathbf{x}) - \bar{f}(\mathbf{x})), \quad \bar{f}(\mathbf{x}) = \sum_j x_j f_j(\mathbf{x}) \quad (10)$$

where x_k is the frequency of strategy k and f_k is the payoff function.

6.4 Facility Siting Optimisation

The p -median facility problem minimises weighted aggregate travel:

$$\min_{\mathbf{y}, \mathbf{x}} \sum_{i \in I} \sum_{j \in J} h_i d_{ij} x_{ij} \quad \text{s.t.} \quad \sum_j y_j = p, \quad \sum_j x_{ij} = 1, \quad x_{ij} \leq y_j \quad (11)$$

where $y_j \in \{0, 1\}$ indicates facility placement and h_i is demand weight.

7 Three-Dimensional Visualisation: VoxCity

VoxCity provides browser-native 3D building extrusion from GeoJSON footprints. Height strategies include attribute-driven ($h = a_v$ from a specified field), floor-count estimation ($h = n_{\text{floors}} \times h_0$), and land-use rule defaults. CityJSON v2.0 import supports semantic surface classification (WallSurface, RoofSurface, GroundSurface) and attribute queries.

The solar simulation hook (`useSunlightSim`) computes surface irradiance for user-specified date d , time t , and latitude ϕ :

$$E_{\text{sur}} = E_0 \cos \theta_z \cdot \tau_{\text{atm}}(m) \quad (12)$$

where $E_0 = 1,361 \text{ W m}^{-2}$ is the solar constant, θ_z is the solar zenith angle (computed from d , t , ϕ), and $\tau_{\text{atm}}(m)$ is the atmospheric transmittance as a function of air mass m . This enables

daylight assessment and urban heat island diagnostics within the Map Explorer layer stack.

8 AI Orchestration and Retrieval-Augmented Generation

8.1 Multi-Provider LLM Orchestration

The AI service layer provides a unified streaming interface over four model providers: OpenAI, Anthropic, Google Gemini, and local Ollama. Provider adapters implement a common contract covering streaming token delivery, error classification, retry scheduling, and usage accounting. A routing layer selects providers based on model capability registries, cost targets, and latency constraints. The streaming phase controller governs state transitions: *idle* → *context-building* → *streaming* → *applying* → *idle*.

8.2 Retrieval-Augmented Generation

The RAG subsystem augments LLM responses with evidence from two sources: workspace documents (uploaded PDFs, markdown notes, analysis outputs) and a built-in domain academic corpus indexed across spatial statistics, urban morphology, transport modelling, environmental analysis, and planning theory. Chunking follows a document-aware strategy respecting section and code-block boundaries. Retrieval uses cosine-similarity search against a flat-array embedding index. Citations are attached in structured format (`[path:fromLine-toLine]`), providing traceable provenance. Guardrails enforce topic scope and require retrieved evidence for specific numeric assertions, falling back to an explicit uncertainty statement below a configurable confidence threshold.

9 Reproducibility Design

Reproducibility is enforced at four architectural levels.

Deterministic seeding. All stochastic operations (LISA permutation tests, Monte Carlo simulation, agent initialisation) accept explicit seeds stored in the workspace manifest. The Muberry32 PRNG is used consistently, producing bit-identical outputs for the same seed.

Manifest-backed artefact traceability. Each analytical output is associated with a manifest entry recording: generating function name, input dataset paths with SHA-256 hashes, parameter snapshot, seed, timestamp, and model version (for AI-generated content). The manifest is exported alongside outputs, enabling independent verification.

Apply-plan history. `useApplyHistoryStore` records each AI-assisted edit as a labelled change-set with before/after state snapshots, allowing selective rollback and revision comparison.

Environment documentation. The workspace configuration snapshot includes provider model identifiers, package versions, and data pipeline configuration, ensuring that AI-generated content is attributable to a specific model checkpoint.

10 Discussion and Limitations

10.1 Contributions

Synapse IDE makes three contributions to computational planning practice. First, it demonstrates that a comprehensive spatial analysis environment can be delivered as a browser-native application without server-side computation dependencies for core functions. Second, the three-panel architecture with bridge connections provides a level of tool integration not available in existing notebook or GIS environments: analytical results propagate directly into cartographic layers, and map selections inject context into AI prompts, within the same session. Third, the domain-specific type system and eight-section document structure provide a methodological scaffold that lowers the barrier for practitioners without deep methodological expertise.

10.2 Current limitations

The GWR and LISA engines are optimised for interactive datasets ($n \leq 5,000$); larger inputs require the Web Worker pathway and may still be slow for very large urban datasets. The VoxCity solar simulation assumes clear-sky conditions. The RAG corpus covers a curated subset of spatial methods literature and does not auto-update from live bibliographic databases. The ABM framework provides a typed scaffold but does not implement specific agent behaviour models. The collaborative multi-user mode is partially implemented and not yet production-ready.

10.3 Future directions

A Python execution bridge will allow Jupyter-compatible notebooks to run within the workbench without leaving the environment. The RAG corpus will be extended with live retrieval from OpenAlex and Semantic Scholar. The WebGPU pipeline will accelerate spatial interpolation and rasterisation for large-scale datasets. Completion of the collaborative mode will enable joint analysis sessions with tracked contributions across institutional boundaries.

11 Conclusion

Synapse IDE: Urban Analytics Workbench integrates spatial statistics, urban simulation, three-dimensional visualisation, and AI assistance within a single browser-native, three-panel environment designed for reproducible urban planning research. By encoding domain knowledge in a structured type system, implementing GWR, LISA, CA, ABM, and facility siting computations in a typed engine layer with Web Worker offloading, and enforcing manifest-based artefact traceability across three integrated panels, the workbench addresses structural limitations of existing fragmented toolchains. The system is open-source, requires no local installation, and is designed to lower the computational planning barrier for researchers and practitioners in data-sparse and resource-constrained urban contexts.

11 References

- Anselin, L. (1995). Local indicators of spatial association—LISA. *Geographical Analysis*, 27(2), 93–115.
- Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 57(1), 289–300.
- Bettencourt, L.M., Lobo, J., Helbing, D., Kühnert, C., & West, G.B. (2007). Growth, innovation, scaling, and the pace of life in cities. *PNAS*, 104(17), 7301–7306.
- Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analysing, and visualising complex street networks. *Computers, Environment and Urban Systems*, 65, 126–139.
- Feng, J. et al. (2025). CityGPT: Empowering urban spatial cognition of large language models. *KDD 2025*.
- Fotheringham, A.S., Brunsdon, C., & Charlton, M. (2002). *Geographically Weighted Regression*. Wiley.
- Getis, A., & Ord, J.K. (1992). The analysis of spatial association by use of distance statistics. *Geographical Analysis*, 24(3), 189–206.
- Hillier, B., & Hanson, J. (1984). *The Social Logic of Space*. Cambridge University Press.
- Hurvich, C.M., Simonoff, J.S., & Tsai, C.L. (1998). Smoothing parameter selection in non-parametric regression using an improved Akaike information criterion. *Journal of the Royal Statistical Society, Series B*, 60(2), 271–293.
- Ledoux, H. et al. (2019). CityJSON: A compact and easy-to-use encoding of the CityGML data model. *Open Geospatial Data, Software and Standards*, 4(4).
- Lewis, P. et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.
- Li, Z. et al. (2023). UrbanGPT: Spatio-temporal large language models. *arXiv:2403.00813*.
- Mai, G. et al. (2023). On the opportunities and challenges of foundation models for geospatial artificial intelligence. *arXiv:2304.06798*.
- Singleton, A., Spielman, S., & Brunsdon, C. (2016). Establishing a framework for open geographic information science. *International Journal of Geographical Information Science*, 30(8), 1507–1521.
- Waddell, P. (2002). UrbanSim: Modeling urban development for land use, transportation, and environmental planning. *Journal of the American Planning Association*, 68(3), 297–314.